紫光展锐 UNISOC

Unisoc Confidential For longcheer

# Android Multimedia 音视频基础概念

WWW.UNISOC.COM

# 修改历史

| 版本号 | 日期 | 注释 |
| --- | --- | --- |
| V1.0 | 2023/06/01 | 第一次正式发布。 |
| | | |
| | | |

Unisoc Confidential For longcheer

Unisoc Confidential For longcheer

# 目录

01 视频、图像像素点数据格式

02 视频逐行、隔行扫描与帧率

03 音频采集相关参数

04 音视频封装

05 Android平台音视频框架简单介绍

06 常用音视频分析工具介绍

紫光展锐 UNISOC

Unisoc Confidential For longcheer

# 01

# 视频、图像像素点数据格式

# 视频、图像像素点数据格式

当我们看视频时会看到很多图像，这些图像由一个个的像素点组成线，又由一条条线组成面，这个面铺在屏幕上展现出来就是我们看到的图像。

这些图像有黑白的，也有彩色的。这是因为图像输出设备支持的规格不同，所以色彩空间也有所不同，不同的色彩空间能展现的色彩明暗程度，颜色范围等也不同。

这里介绍下几种常见的色彩格式。

Huang Longcheng

**1.GRAY 灰度模式表示**

在 20 世纪 80、90 年代，国内大多数家庭看的还是黑白电视。那个黑白电视的图像就是以 GRAY 的方式展现的图像，也就是 Gray 灰度模式。这一模式为 8 位展示的灰度，取值 0 至 255，表示明暗程度，0 为最黑暗的模式，255 为最亮的模式，色彩表示范围如图所示：

由于每个像素点是用 8 位深展示的，所以一个像素点等于占用一个字节，一张图像占用的存储空间大小计算方式也比较简单：

$$图像占用空间 = 图像宽度（W）\times 图像高度（H）\times 1$$

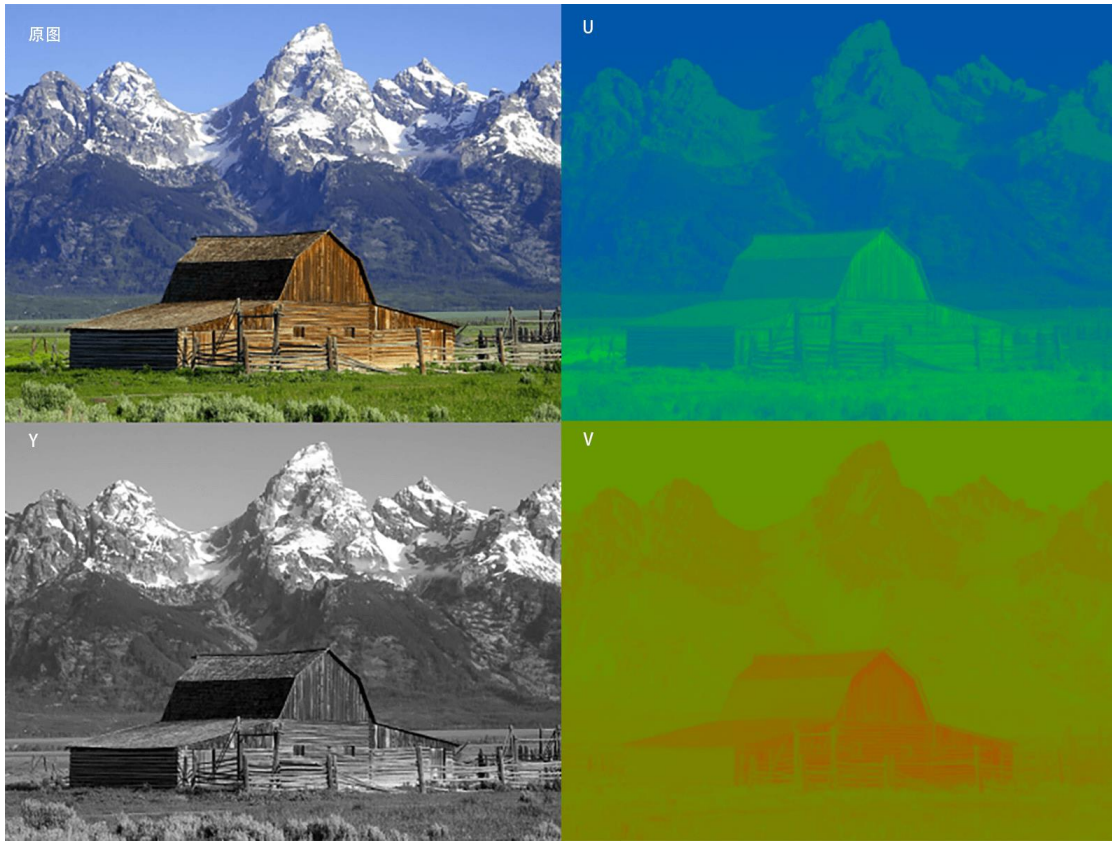
# 视频、图像像素点数据格式

当举个例子，如果图像为 352x288 的分辨率，那么一张图像占用的存储空间应该是 352x288，也就是 101376 个字节大小。

Unisoc Confidential For longcheer

**2.YUV 色彩表示**

在视频领域，通常以 YUV 的格式来存储和显示图像。其中 Y 表示视频的灰阶值，也可以理解为亮度值，而 UV 表示色彩度，如果忽略 UV 值的话，我们看到的图像与前面提到的 GRAY 相同，为黑白灰阶形式的图像。YUV 最大的优点在于每个像素点的色彩表示值占用的带宽或者存储空间非常少。



# 视频、图像像素点数据格式

为节省带宽起见，大多数 YUV 格式平均使用的每像素位数都少于 24 位。主要的色彩采样格式有 YCbCr 4：2：0、YCbCr 4：2：2、YCbCr 4：1：1 和 YCbCr 4：4：4。YUV 的表示法也称为 A：B：C 表示法。

为了便于理解，下面以 352x288 的图像大小为例，来详细介绍一下各个采样格式的区别。

Unisoc Confidential For longchee

**2.1. YUV 4：4：4 格式**

yuv444 表示 4 比 4 比 4 的 yuv 取样，水平每 1 个像素（即 1x1 的 1 个像素）中 y 取样 1 个，u 取样 1 个，v 取样 1 个，所以每 1x1 个像素 y 占有 1 个字节，u 占有 1 个字节，v 占有 1 个字节，平均 yuv444 每个像素所占位数为：

$$(1+1+1) \times 8bit \div 1pix = 24bpp$$

那么 352x288 分辨率的一帧图像占用的存储空间为：

$$352 \times 288 \times 24\ (bit) \div 8\ (bit) = 304128\ (字节)$$

# 视频、图像像素点数据格式

紫光展锐 UNISOC

## 2.2. YUV 4：2：2 格式

yuv422 表示 4 比 2 比 2 的 yuv 取样，水平每 2 个像素（即 2x1 的 2 个像素）中 y 取样 2 个，u 取样 1 个，v 取样 1 个，所以每 2x1 个像素 y 占有 2 个字节，u 占有 1 个字节，v 占有 1 个字节，平均 yuv422 每个像素所占位数为：

Unisoc Confidential For kangcheer

$$（2+1+1）\times 8bit \div 2pix = 16bpp$$

那么 352x288 分辨率的一帧图像占用的存储空间为：

$$352 \times 288 \times 16（bit）\div 8（bit）= 202752（字节）$$

# 视频、图像像素点数据格式

紫光展锐 UNISOC

**2.3. yuv4：1：1 格式**

yuv411 表示 4 比 1 比 1 的 yuv 取样，水平每 4 个像素（即 4x1 的 4 个像素）中 y 取样 4 个，u 取样 1 个，v 取样 1 个，所以每 4x1 个像素 y 占有 4 个字节，u 占有 1 个字节，v 占有 1 个字节，平均 yuv411 每个像素所占位数为：

Unisoc Confidential For kangcheer

$$（4+1+1）\times 8bit \div 4pix = 12bpp$$

那么 352x288 分辨率的一帧图像占用的存储空间为：

$$352 \times 288 \times 12（bit）\div 8（bit）= 152064（字节）$$

# 视频、图像像素点数据格式

紫光展锐 UNISOC

**2.4. yuv4：2：0 格式**

yuv420 表示 4 比 2 比 0 的 yuv 取样，水平每 2 个像素与垂直每 2 个像素（即 2x2 的 2 个像素）中 y 取样 4 个，u 取样 1 个，v 取样 1 个，所以每 2x2 个像素 y 占有 4 个字节，u 占有 1 个字节，v 占有 1 个字节，平均 yuv420 每个像素所占位数为：

Unisoc Confidential For kangcheer

$$（4+1+1）\times 8bit \div 4pix = 12bpp$$

那么 352x288 分辨率的一帧图像占用的存储空间为：

$$352 \times 288 \times 12（bit）\div 8（bit）= 152064（字节）$$

# 视频、图像像素点数据格式

为了方便理解 YUV 在内存中的存储方式，我们以宽度为 6、高度为 4 的 yuv420 格式为例，一帧图像读取和存储在内存中的方式如图：

Unisoc Confidential For longcheer

单帧 yuv420：

在字节流中的位置：

# 视频、图像像素点数据格式

YUV420分为YUV420P 和 YUV420SP

Unisoc Confidential For longcheer

**2.4.1. YUV420P**

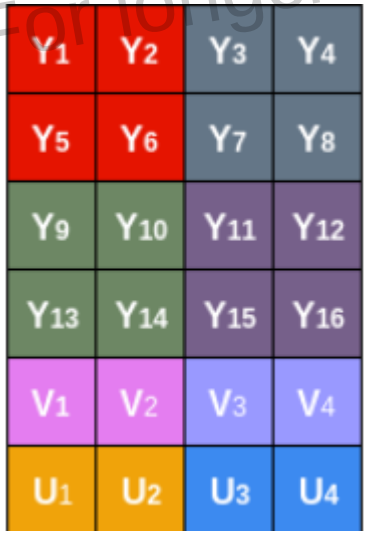
YUV420P又称为plane平面模式，Y、U、V分别在不同的平面，也就是有三个平面，它是YUV标准格式4:2:0，主要分为YU12和YV12.

YU12， 按Y、U、V的顺序存储。

# 视频、图像像素点数据格式

YV12， 按Y、V、U的顺序存储。

Unisoc Confidential For longcheer




### 2.4.1. YUV420SP

YUV420SP格式的图像阵列，首先是所有Y值，然后是UV或者VU交替存储，NV12和NV21属于YUV420SP格式，是一种two-plane模式，即Y和UV分为两个plane，但是UV(CbCr)为交错存储，而不是像YUV420格式那样分为三个平面。主要分为：NV21和NV12

# 视频、图像像素点数据格式

NV21

存储顺序是先存Y值，再VU交替存储。

Unisoc Confidential For longcheer

| | | | |
|---|---|---|---|
| $Y_1$ | $Y_2$ | $Y_3$ | $Y_4$ |
| $Y_5$ | $Y_6$ | $Y_7$ | $Y_8$ |
| $Y_9$ | $Y_{10}$ | $Y_{11}$ | $Y_{12}$ |
| $Y_{13}$ | $Y_{14}$ | $Y_{15}$ | $Y_{16}$ |
| $V_1$ | $U_1$ | $V_2$ | $U_2$ |
| $V_3$ | $U_3$ | $V_4$ | $U_4$ |

# 视频、图像像素点数据格式

紫光展锐 UNISOC

NV12

Unisoc Confidential For longcheer

存储顺序是先存Y值，再UV交替存储。

注：我们平台的视频解码后输出YUV格式设置的是NV12，在查看dump的数据时，要选择正确的格式。

/vendor/sprd/modules/media/codec2/components/video/avc/dec/C2UnisocAvcDec.cpp

```
469            video_format.yuv_format = YUV420SP_NV12;
```

# 视频、图像像素点数据格式

紫光展锐 UNISOC

**3.RGB色彩表示**

三原色光模式（RGB color model），又称 RGB 颜色模型或红绿蓝颜色模型，是一种加色模型，将红（Red）、绿（Green）、蓝（Blue）三原色的色光按照不同的比例相加，来合成各种色彩光。

Unisoc Confidential For Longcheer

每象素 24 位编码的 RGB 值：使用三个 8 位无符号整数（0 到 255）表示红色、绿色和蓝色的强度。这是当前主流的标准表示方法，用于交换真彩色和 JPEG 或者 TIFF 等图像文件格式里的通用颜色。它可以产生一千六百万种颜色组合，对人类的眼睛来说，其中有许多颜色已经无法确切地分辨了。

RGB 常见的展现方式分为 16 位模式和 32 位模式（32 位模式中主要用其中 24 位来表示 RGB）。16 位模式（RGB565、BGR565、ARGB1555、ABGR1555）分配给每种原色各为 5 位，其中绿色为 6 位，因为人眼对绿色分辨的色调更敏感。但某些情况下每种原色各占 5 位，余下的 1 位不使用或者表示 Alpha 通道透明度

32 位模式（ARGB8888），实际就是 24 位模式，余下的 8 位不分配到象素中，这种模式是为了提高数据处理的速度。同样在一些特殊情况下，在有些设备中或者图像色彩处理内存中，余下的 8 位用来表示象素的透明度（Alpha 通道透明度）

紫光展锐 UNISOC

Unisoc Confidential For longcheer

# 02

# 视频逐行、隔行扫描与帧率

# 视频逐行、隔行扫描与帧率

当我们看到一些老电视剧、老电影或者一些 DV 机拍摄的视频时，会发现视频中物体在移动时会出现条纹，这些条纹的存在主要是因为视频采用了隔行扫描的刷新方式。

Unisoc Confidential For Longcheer

**1.隔行扫描与逐行扫描**

隔行扫描（Interlaced）是一种将图像隔行显示在扫描式显示设备上的方法，例如早期的 CRT 电脑显示器。非隔行扫描的扫描方法，即逐行扫描（Progressive），通常从上到下地扫描每帧图像，这个过程消耗的时间比较长，占用的频宽比较大，所以在频宽不够时，很容易因为阴极射线的荧光衰减在视觉上产生闪烁的效应。而相比逐行扫描，隔行扫描占用带宽比较小。扫描设备会交换扫描偶数行和奇数行，同一张图像要刷两次，所以就产生了我们前面说的条纹。

# 视频逐行、隔行扫描与帧率

早期的显示器设备刷新率比较低，所以不太适合使用逐行扫描，一般都使用隔行扫描。在隔行扫描的时候，我们常见的分辨率描述是 720i、1080i，“i”就是 Interlaced。现在我们看视频播放器相关广告和说明时，还经常会看到 720p、1080p 这样的说法，这个“p”又代表什么呢?

Unisoc Confidential For Internal Use

在如今这个时代的显示器和电视中，由于逐行扫描显示的刷新率的提高，使用者已经不会感觉到屏幕闪烁了。因此，隔行扫描技术逐渐被取代，逐行扫描越来越常见，也就是我们经常见到的 720p、1080p。

**2.帧率**

就是我们平时提到的帧率（FrameRate），指一秒钟刷新的视频图像帧数（Frames Per Second），视频一秒钟可以刷新多少帧，取决于显示设备的刷新能力。不同时代的设备，不同场景的视频显示设备，刷新的能力也不同，所以针对不同的场景也出现了很多种标准，例如：

1).NTSC 标准的帧率是 30000/1001，大约为 29.97 fps
2).PAL 标准的帧率是 25/1，为 25 fps
3).QNTSC 标准的帧率是 30000/1001，大约为 29.97 fps

# 视频逐行、隔行扫描与帧率

紫光展锐 UNISOC

4).QPAL 标准的帧率是 25/1，为 25 fps
5).SNTSC 标准的帧率是 30000/1001，大约为 29.97 fps
6).SPAL 标准的帧率是 25/1，为 25 fps
7).FILM 标准的帧率是 24/1，为 24 fps
8).NTSC-FILM 标准的帧率是 24000/1001，大约为 23.976 fps

Unisoc Confidential For Longcheer

如果用心观察的话，你会发现 NTSC 标准的分辨率都不是整除的帧率，分母都是 1001，为什么会这样呢?

NTSC 制式的标准为了解决因为色度和亮度频率不同引起失真色差的问题，将频率降低千分之一，于是就看到了有零有整的帧率。我们在电影院看的电影的帧率，实际上标准的是 23.97 fps，所以我们可以看到给院线做视频后期制作的剪辑师们最终渲染视频的时候，大多数会选择 23.97 fps 的帧率导出。

# 视频逐行、隔行扫描与帧率

紫光展锐 UNISOC

**3.图像分辨率**

当人们在谈论流畅、标清、高清、超高清等清晰度的时候，其实主要想表达的是分辨率。它是衡量图像细节表现力的重要的技术参数。

Unisoc Confidential For Longcheer

除了分辨率之外，我们还需要结合视频的类型、场景等设置适合的码率（单位时间内传递的数据量）。一般，分辨率越高代表图像质量越好，越能看到图像的更多细节，文件也就会越大。分辨率通常由宽、高与像素点占用的位数组成，计算方式为图像的宽乘以高。

我们经常听到人们提到 1080p、4K，其实它们还有更标准的称呼或者叫法，例如 1080p 我们又叫 Full-HD，通常接近 4K 的分辨率我们叫 4K 也没太大问题，像有更标准的叫法，比如 3840x2160 的分辨率应该是 UHD-1。但是如果直接按标准叫法来叫的话，国内很多人可能不太习惯，为了便于区分，通常就直接说分辨率的宽乘以高的数值。因为 4k 的表述比较简洁，所以就可以模糊地说是 4K 了。

紫光展锐 UNISOC

Unisoc Confidential For longcheer

# 03

# 音频采集相关参数

# 音频采集相关参数

紫光展锐 UNISOC

## 1.音频采样数据格式

Unisoc Confidential For Longcheer

介绍音频采样数据格式之前，我们需要先了解音频从采集一直到我们耳朵听到声音这个过程中都发生了什么，我们先看一下下面这张流程图：

首先我们说的话或者在自然界中听到的一些声音，比如鸟鸣，水流等，都是通过空气振动来传输的模拟信号，我们可以通过麦克风或者拾音器采集到声音的模拟信号，然后将模拟信号转换成数字信号，这个过程可以通过麦克风来做，也可以通过音频的转换器来做，转换成数字信号之后将数字信息存储起来，或者输出到扬声器，扬声器会根据数字信号产生一定频率的振动，然后通过空气传播模拟信号到我们的耳朵里面，我们就听到了对应的声音

# 音频采集相关参数

紫光展锐 UNISOC

在这个流程里我们需要了解一个基本的操作，就是先采集到模拟信号，然后通过 ADC（模数转换）将模拟信号转换成数字信号以后，再通过 PCM（Pulse Code Modulation）脉冲编码调制对连续变化的模拟信号进行采样、量化和编码转换成离散的数字信号，从而实现音频信号的采集。另外，也可以将采集的音频信号输出到扬声器、耳机之类的设备。

Unisoc Confidential For Internal Use

我们上面说的 PCM 文件就是未经封装的音频原始文件，或者叫做音频“裸数据”。不同的扬声器、耳机设备，甚至是声卡输出设备，对音频的裸数据支持的情况不一样，有的设备支持单精度浮点型数据、有的设备支持双精度浮点型数据、有的设备支持无符号型数据、有的设备支持有符号型数据。因为输出的数据类型的支持不同，所以 PCM 采样数据的格式在输出之前，需要转换一下。这些数据的格式我们通常称之为采样数据格式。

# 音频采集相关参数

紫光展锐 UNISOC

**2.音频采样率**

音频 PCM 数据的输入和输出是需要有一个频率的，频率通常在我们听觉可接受的范围内，太高或者太低我们都听不见，通常我们人耳能够听到的频率范围是在 20Hz～20kHz 之间，为了保证音频不失真，音频的采样频率通常应该在 40kHz 以上，而理论上采样率大于 40kHz 的音频格式都可以称之为无损格式。现在一般的专业设备的采样频率为 44100Hz（也称之为 44.1kHz）。并且 44.1kHz 是专业音频中的最低采样率。当然要听到更高采样率，比如 96kHz、192kHz 采样频率中的细节的话，就取决于耳朵和对应的设备了。

Unauthorized copy for longcheer

下面简单地介绍一下，在数字音频领域常用的采样率与对应的使用场景：

1).8000 Hz 主要是电话通信时用的采样率，对于传达人们说话时的声音已经足够了

2).11025 Hz、22050 Hz 主要是无线电广播用的采样率

3).44100 Hz 常用于音频 CD，MP3 音乐播放等场景

4).48000 Hz 常用于 miniDV、数字电视、DVD、电影和专业音频等设备中

# 音频采集相关参数

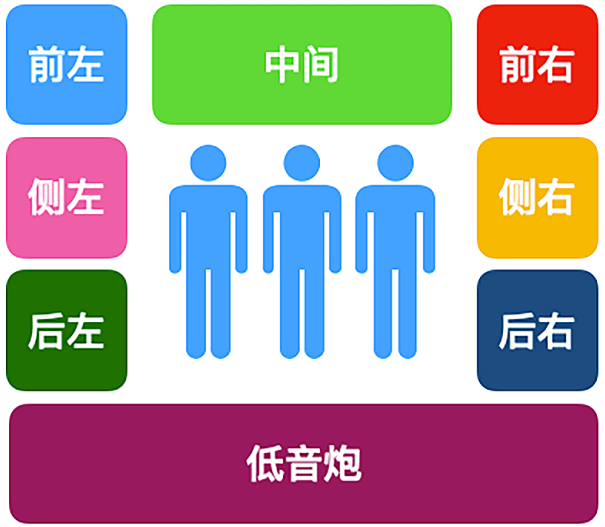
紫光展锐 UNISOC

**3.音频声道及其布局**

当我们戴着耳机看电视剧、电影、听音乐、开会的时候，会发现左右耳朵听到的声音有时候会有一些差别，尤其是当我们看 TVB 港剧的时候，一只耳朵听到的是粤语，另一只耳朵听到的是普通话。因为视频里面的音频支持左声道右声道内容不同，所以有些时候我们看电影可以切换左声道右声道模式。

Unisoc Confidential For longcheer

采集不同方位的声源，然后通过不同方位的扬声器播放出来就产生了不同的声道。其实我们常见的声道内容除了左声道、右声道，还有立体声等，当我们听到的音频声道比较多，比如听交响乐的时候，立体感会尤为明显，示意图如下:



# 音频采集相关参数

紫光展锐 UNISOC

**4.音频采样位深度**

采样的位深度，也叫采样位深，它决定了声音的动态范围。平时，我们常见的 16 位（16bit）可以记录大概 96 分贝（96dB）的动态范围。也可以理解为每一个比特大约可以记录 6dB 的声音

Unisoc Confidential For Longcheer

通常为了高保真，我们会选择使用 32bit，甚至 64bit 来表示音频。而常规音频通话使用 16bit 来表示即可，当然条件有限的话，8bit 也可以，但它是底线。因为 8bit 的音频表示，听起来有时候会比较模糊。

# 音频采集相关参数

紫光展锐 UNISOC

**5.音频的码率**

接下来我们看一下音频的码率。所谓码率，我们通常可以理解为按照某种频率计算一定数据量的单位，重点体现在“率”上面，我们常用的码率统计时间单位为秒，所以码率也就是一秒钟的数据量，通常我们用 bps（bits per second）来表示，也就是每秒钟有多少位数据。而我们平时所说的码率，可以简单理解为每秒钟存储或传输的编码压缩后的数据量。

Unisoc Confidential For Longcheer

那这个音频码率是怎么算出来的呢？其实这里有一个公式，可以帮助我们算出音频的码率。例如我们有一个双声道立体声、采样率是 48000、采样位深是 16 位、时长为 1 分钟的音频，它的存储空间占用计算应该是：

声道数×采样率×采样位深×时长=2×48000×16×60=92160000b=11520000B=11.52MB

码率应该是：

92160000b÷60s=1536000bps=1536kbps=1.536Mbps

音频的码率可以间接地表示音频的质量，一般高清格式的码率更高。

# 音频采集相关参数

紫光展锐 UNISOC

**6.音频的编解码**

我们在传输音频文件的时候经常会看到文件名后面有 MP3、AAC 这样的后缀，其实这些都是音频编码的格式。因为音频在传输和存储时，如果直接存储 PCM 音频数据的话，消耗的带宽或者存储空间会比较多，所以我们为了节省传输带宽或者存储，通常会选择对音频数据做编码压缩处理。

Unisoc Confidential For Yongcheer

我们在互联网上常见的音频编码有 AAC、MP3、AC-3、OPUS，个别地方还可能会使用 WMA，但是从兼容性来看，AAC 和 OPUS 更出众一些。目前 AAC 应用于众多音乐播放器和音乐格式封装中，OPUS 常见于语音通信中。

紫光展锐 UNISOC

Unisoc Confidential For longcheer

# 04

# 音视频封装

# 音视频封装

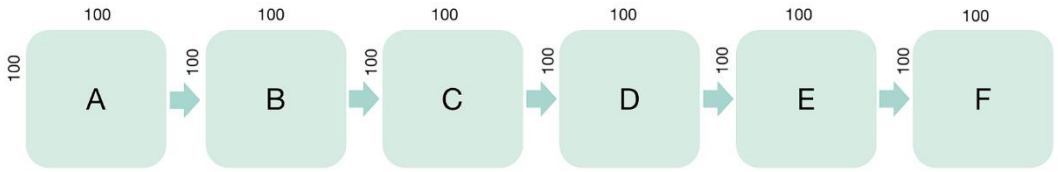
紫光展锐 UNISOC

## 1.音视频编解码

Unisoc Confidential For Lynacheer

我们先来看音视频编码方面的基础知识。就像前面介绍的，音频的 PCM 数据、视频的 YUV 和 RGB 的图像数据直接存储或者通过带宽传输会比较消耗空间，那么为了节省存储，或者确保占用的带宽更少一些，就需要把音频、视频的数据压缩一下。

音频是连续的采样序列，而视频则是连续的图像序列，这些序列是有前后关系的，为了方便理解，我们用图像来举例，看上去会更直观一些。




我们有一个 6 帧的连续视频图像，每一帧都是宽 100、高 100 的图幅，在每一帧的正中央都有一个字母在变化。遇到这种情况时，如果我们每一帧图像全都做传输或存储操作的话，占用的带宽或空间都会很大。

极客时间

# 音视频封装

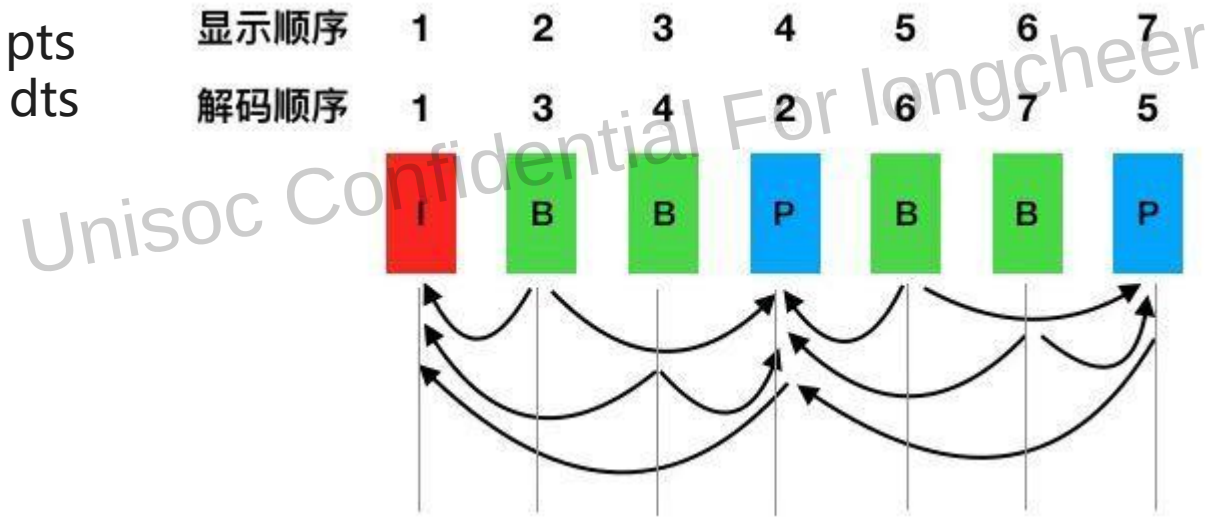
为了节省空间，我们可以来分析一下图像的规律。因为这 6 帧图像大范围是相同的，只有正中心的一小部分内容是变化的，所以我们分析后可以得到结论，刷新第一帧图像后，从第二帧开始，我们只要刷新正中心字母区域的内容即可。这个叫局部更新，只需要逐步更新 A、B、C、D、E、F 的区域就可以了，这样在传输内容的时候既节省带宽，又能在存储内容的时候节省画幅的数据存储空间。

Unisoc Confidential For Internal Use

而我们在看视频的时候可不仅仅是内容的更新，还涉及内容位置的运动等变化，所以视频内容更新的算法会更复杂一些。在做视频压缩的时候，就拿前面的这个例子来说，需要有一个参考帧，这里参考的是第一帧，后面每一帧都参考前面一帧做了局部更新。而我们的视频图像序列不能只做局部更新，因为里面的目标对象还会运动，所以我们不仅可以前向做参考，还有可以做双向参考的技术。在这个过程中就涉及了图像的类型，通常我们遇到的是这三类帧：I 帧、P 帧和 B 帧。

其中 I 帧作为关键帧，仅由帧内预测的宏块组成。而 P 帧代表预测帧，通过使用已经编码的帧进行运动估计，B 帧则可以参考自己前后出现的帧。如果比较 IBP 帧包所占空间大小的话，通常是 I 帧 > P 帧 > B 帧，所以适当地增加 B 帧可以减少视频流占用的带宽或者存储空间。我们可以看一下，I、P、B 帧在视频解码显示时的顺序。

# 音视频封装




Unisoc Confidential For longcheer

从这个示意图里，我们可以看出，解码顺序是 1423756，但是显示顺序却是 1234567。当编码中存在 B 帧的时候，因为解码需要双向参考帧，所以需要多缓存几帧作为参考数据，从而也就带来了一定的显示延迟。所以在实时直播场景下，参考标准中推荐的做法通常是不带 B 帧。

在视频编码时，因为图像的画面以及图像中对象运动的复杂程度比较高，为了保证清晰度，运动的图像组中通常也会包含更多的图像运动参考信息，所以压缩难度也提升了很多，压缩后的视频码率也就变得比常规图像更高一些，这个码率的波动通常时高时低，具有可变性，我们一般称之为可变码率（VBR）。

# 音视频封装

而在有些直播场景下，因为一个传输信号通道中会携带多条流，为了确保多条流在同一个信号通道中互不干扰，一般会要求编码时采用恒定码率的方式（CBR）。但是如果采用 CBR 的话，画质往往会有一些损耗，这也就是为什么我们现在在一些老式的电视直播场景中看到的画质偶尔会变得很差，比如交通广播电视场景。

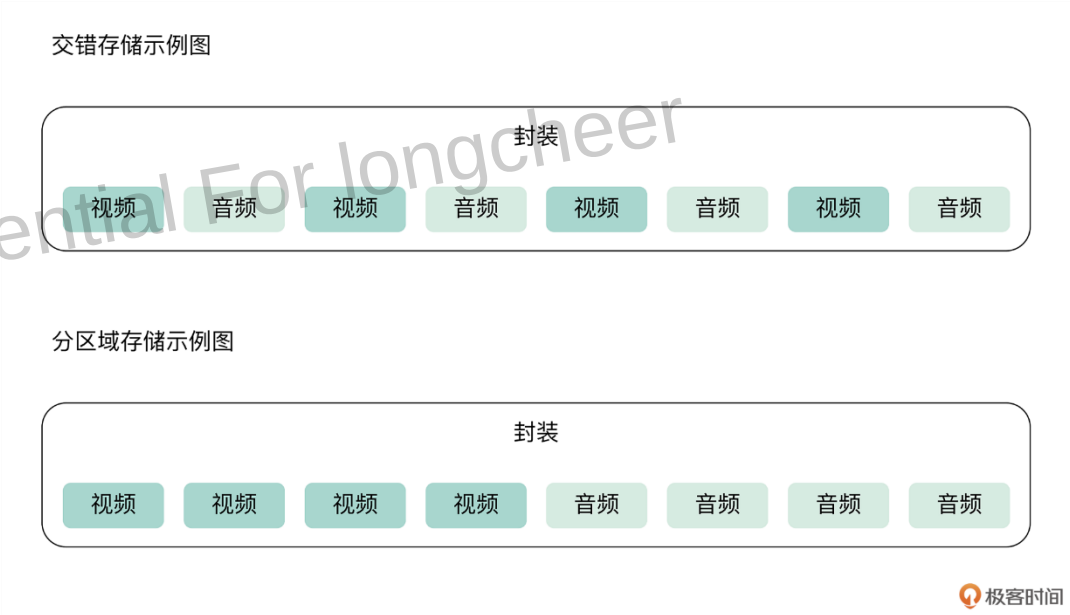
Unisoc Confidential For Engineer

**2. 视频封装**

我们平时经常会听到有人说自己的视频是 MP4 格式的，有些人也会说自己的音乐格式是 M4A，这些都是什么意思呢?

其实我们可以粗略地认为他们说的是封装格式，也就是常说的容器格式。在容器格式的内部会存储音频、视频的数据，这些数据我们可以称之为视频流、音频流。音视频流在容器中的存储形式有两种，既可以交错式存储，也可以是不同类型的流单独存储在自己的连续区域。

# 音视频封装




Unisoc Confidential For longcheer

**2. 封装容器格式: MP4**

在互联网常见的格式中，跨平台最好用的应该是 MP4 文件，因为 MP4 文件既可以在 PC 平台的 Flashplayer 中播放，又可以在移动平台的 Android、iOS 等平台中播放，而且使用系统默认的播放器就能播放，因此我们说 MP4 格式是最常见的多媒体文件格式，接下来我们就来了解一下 MP4 这种文件格式。

1).MP4 文件由许多个 Box 与 FullBox 组成
2).每个 Box 由 Header 和 Data 两部分组成
3).FullBox 则是 Box 的扩展，在 Box 结构的基础上，在 Header 中增加 8bit 位 version 标志和 24bit 位的 flags 标志

# 音视频封装

紫光展锐 UNISOC

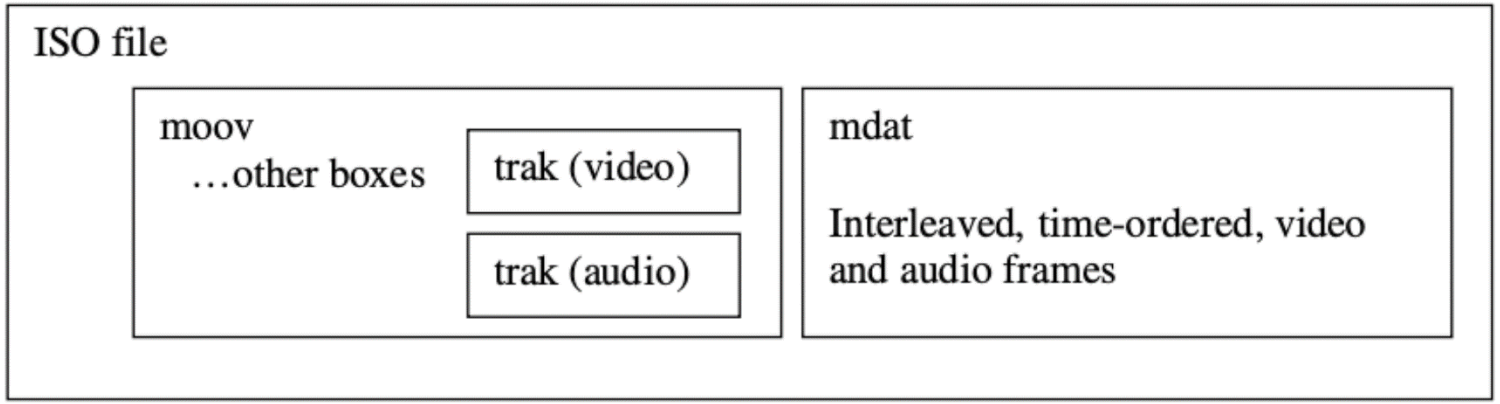
Unisoc Confidential For Langcheer

4).Header 包含了整个 Box 的长度的大小（Size）和类型（Type），当 Size 等于 0 时，代表这个 Box 是文件中的最后一个 Box,当 Size 等于 1 时说明 Box 长度需要更多的 bits 位来描述，在后面会定义一个 64bits 位的 largesize 用来描述 Box 的长度,当 Type 为 uuid 时，说明这个 Box 中的数据是用户自定义扩展类型

5).Data 为 Box 的实际数据，可以是纯数据，也可以是更多的子 Box

6).当一个 Box 中 Data 是一系列的子 Box 时，这个 Box 又可以称为 Container Box。

MP4 封装格式文件中，我们经常会遇到 moov box 与 mdat box。我们存储音频与视频数据的索引信息，就是存在 moov box 中，音频和视频的索引信息在 moov 中分别存储在不同的 trak 里面，trak 里面会保存对应的数据采样相关的索引信息，通过获得 moov 中的索引信息之后，根据索引信息我们才能从 mdat 中读取音视频数据，所以 MP4 文件中必不可少的是 moov 信息，如果缺少 moov 信息的话，这个点播文件将无法被成功打开。



# 音视频封装

紫光展锐 UNISOC

我们刚才介绍视频封装的时候有提到过，音视频数据存储的时候有交错存储的方式也有音频存在一个区域，视频存在一个区域的方式。其实两种方式都比较常见，但是需要注意一个问题，我们在存储音视频数据的时候，如果是顺序读取音视频数据的话，当然就是音视频数据交错存储比较好，这样会给内存、硬盘或者网络节省很多开销。

Unisoc Confidential For Internal Use

如果音视频分开，单独存放在各自的区域的话，为了更好地做音视频同步，通常会读取一段视频帧数据，再读取一段音频采样数据，这样势必会不断跳跃式地读取硬盘中的数据，或者不断地发起网络请求，如果是 http 请求的话，我们会看到大量的 http range 请求，给网络开销与服务器并发造成很大的压力。

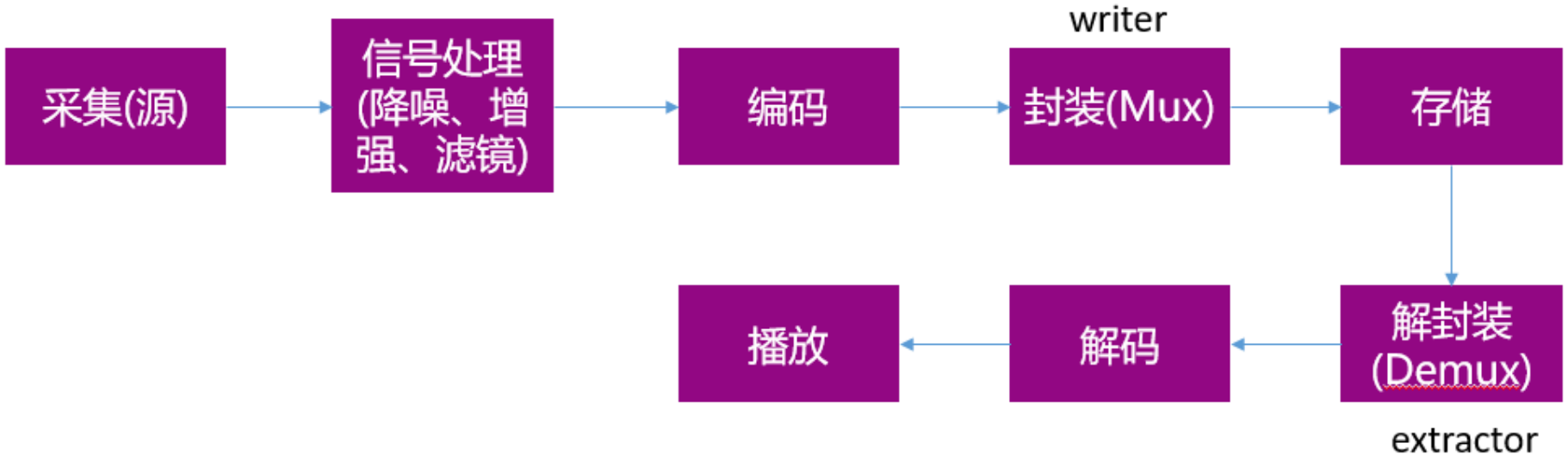
当我们基于网络请求播放 MP4 点播文件时，如果 moov box 存储在 mdat box 后面的话，播放器需要先读取到 MP4 文件的 moov 以后，才能够开始播放 MP4 文件，而这个读取的动作，有些播放器是选择先下载全部 MP4 文件，有些则是需要先解析一下 mdat，跳过 mdat 以后再读取 moov，所以为了节省播放器操作，兼容性更好，通常我们需要将 moov 移到 MP4 文件的头部，节省播放 MP4 文件开始时间段的开销

这里我们需要注意的是，这个 moov 的生成，一般需要先写入 mdat 中的音视频数据，这样我们就知道数据采样存储的位置和大小，然后才能写入到 moov 中，所以是先写入 mdat 后写入 moov 这样一个顺序。解决办法是生成文件之后做一个后处理，也就是将 moov 移动到 mdat 前面。

# 音视频封装

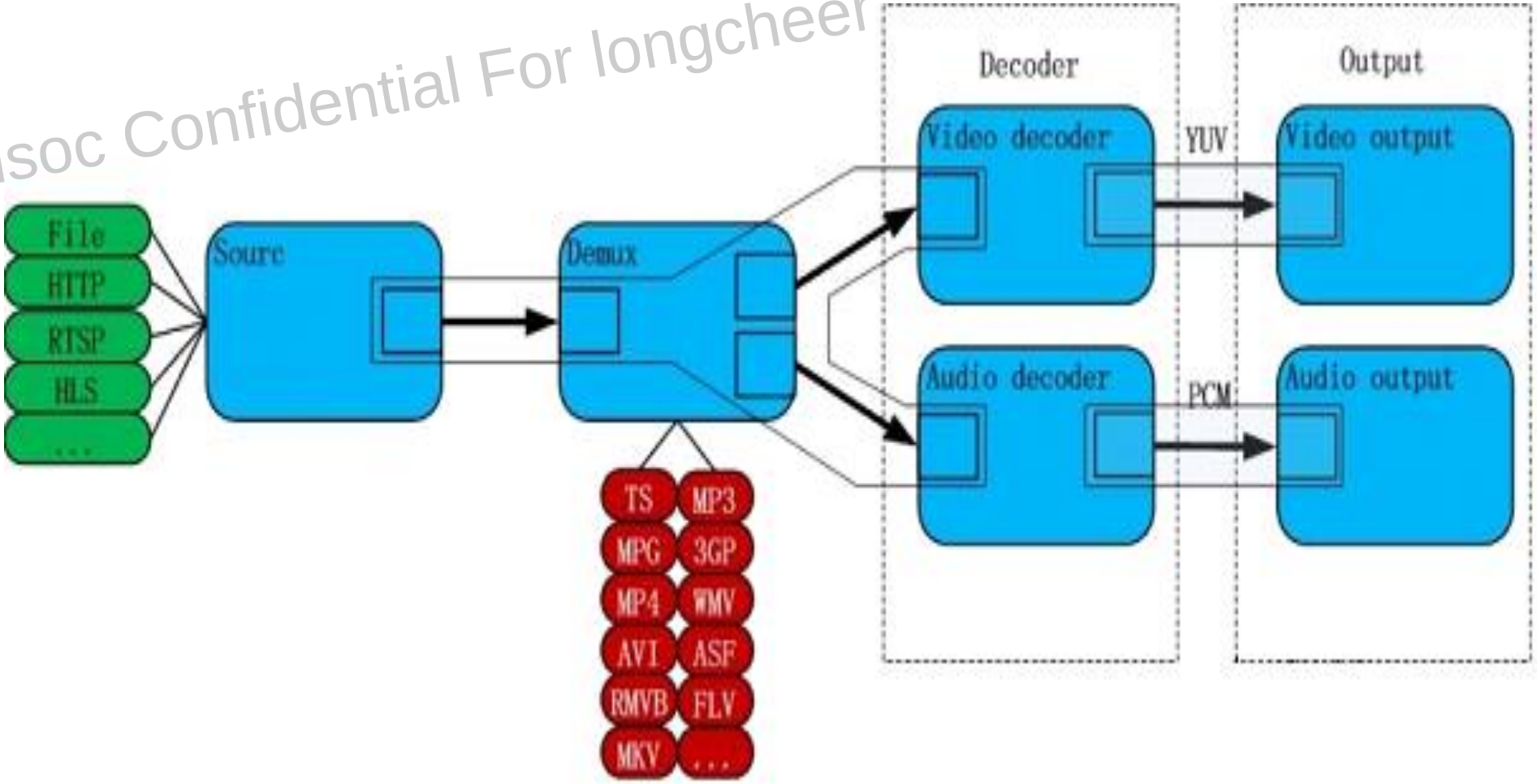
音视频处理基本流程

Unisoc Confidential For longcheer



# 音视频封装

Unisoc Confidential For longcheer



紫光展锐 UNISOC

Unisoc Confidential For longcheer

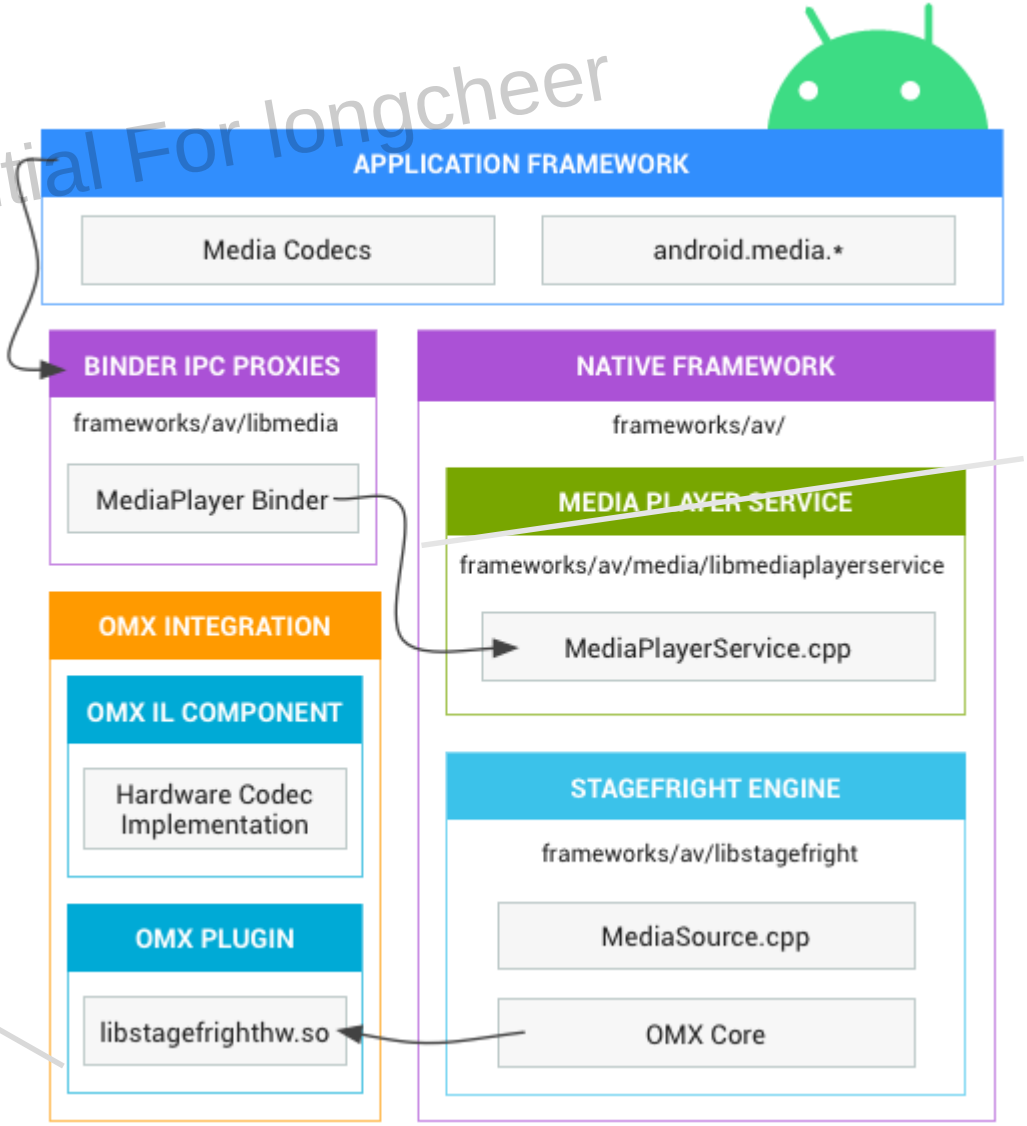
# 05 Android平台音视频框架简单介绍

# Android平台音视频框架简单介绍

紫光展锐 UNISOC

框架图(未包含Codec2)

Unisoc Confidential For longcheer




MediaPlayer的Client端和Server端通过IPC进行通讯

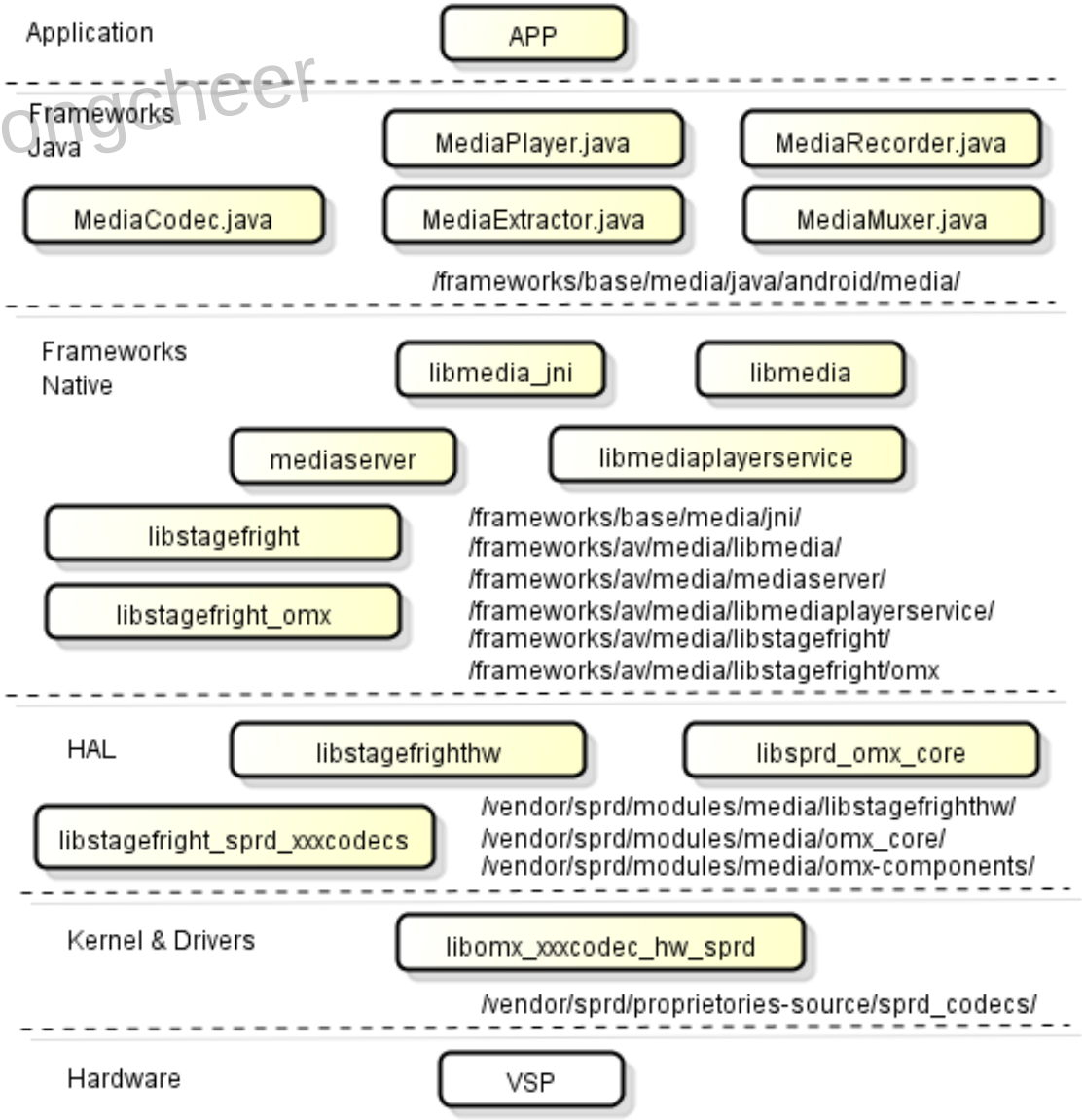
Unisoc解码器组件以插件的方式集成

# Android平台音视频框架简单介绍

紫光展锐 UNISOC

代码层级结构(未包含Codec2，老版omx架构)

Unisoc Confidential For longcheer

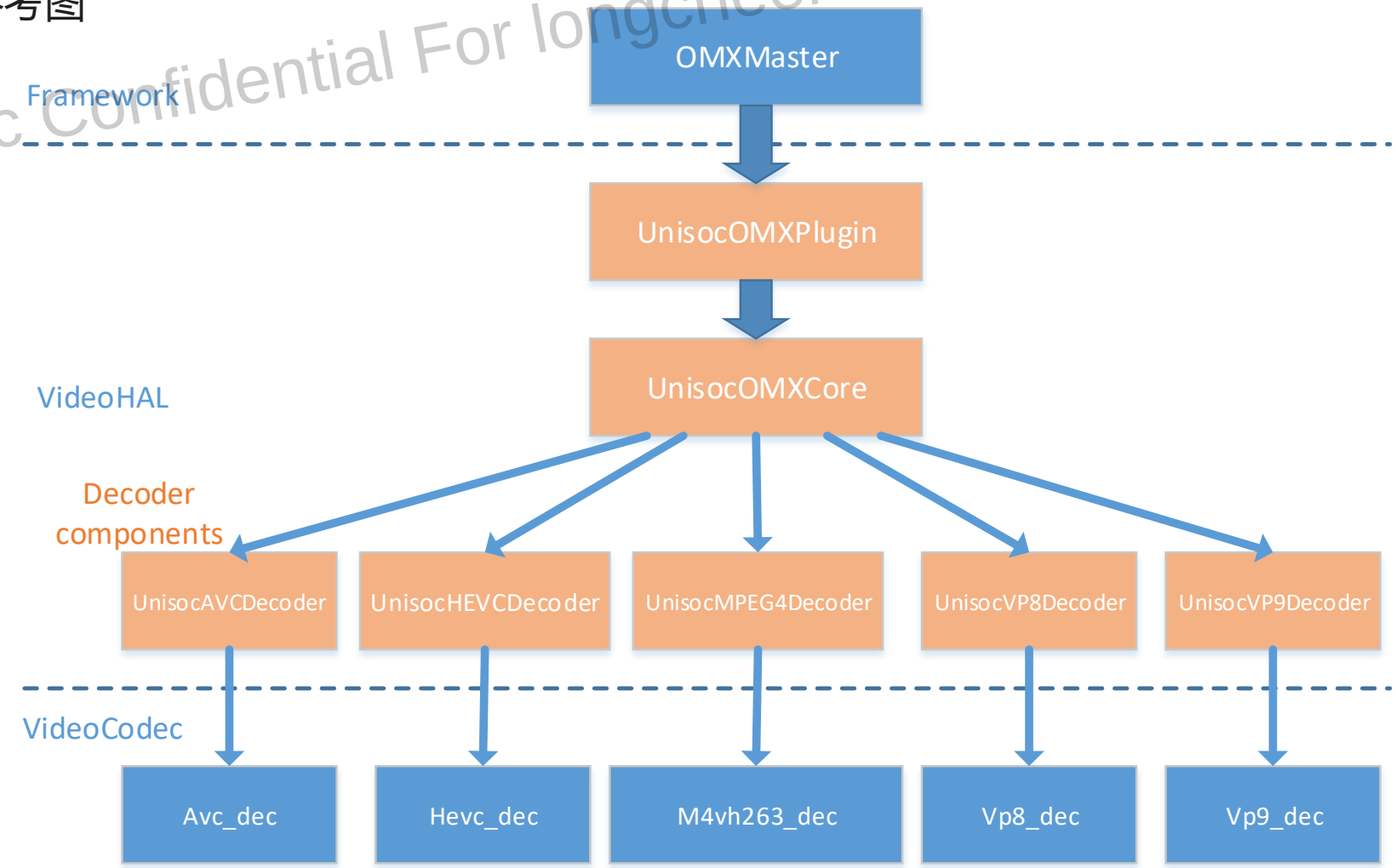


# Android平台音视频框架简单介绍

紫光展锐 UNISOC

Omx组件框图参考图

Unisoc Confidential For longcheer

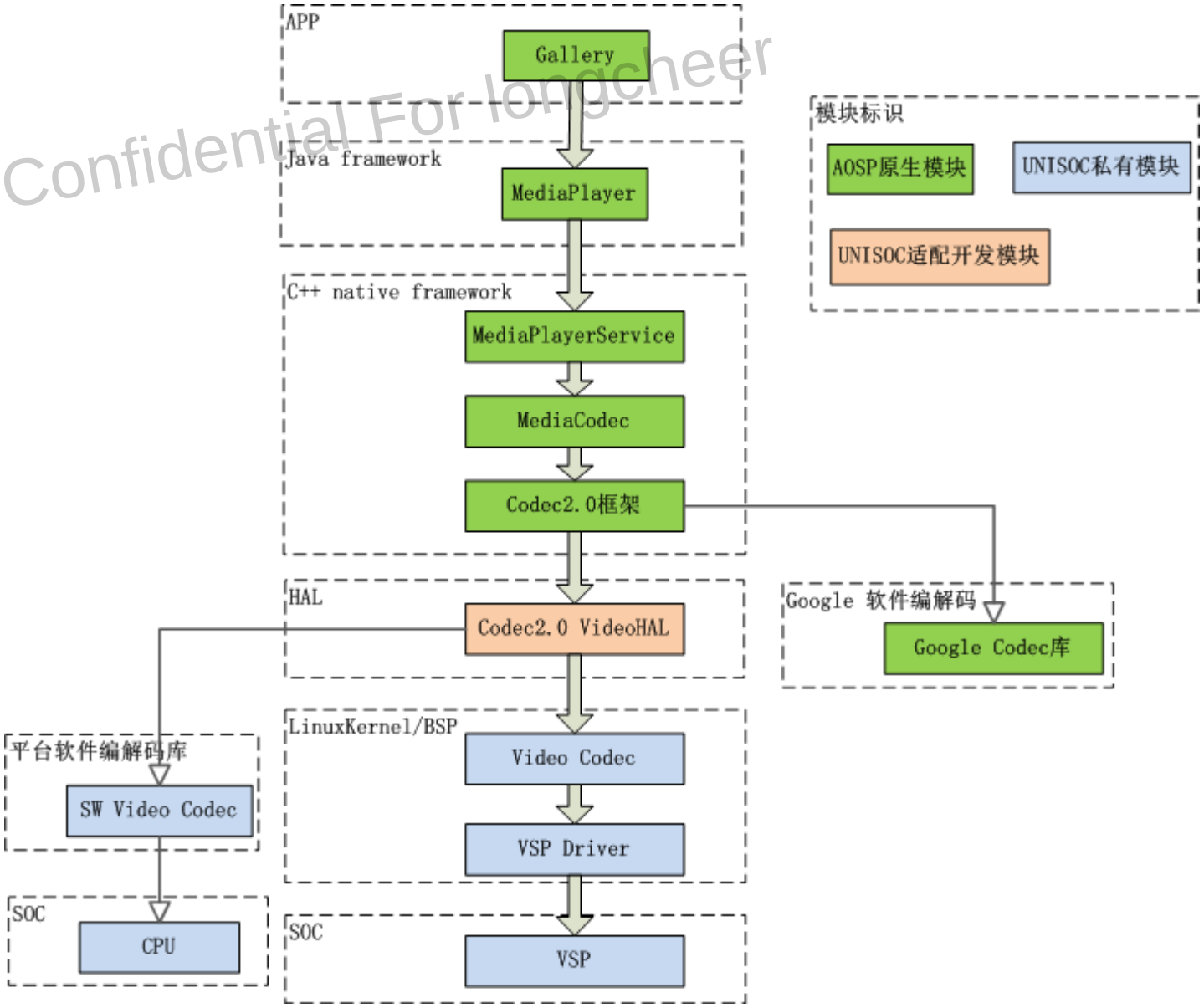


# Android平台音视频框架简单介绍

紫光展锐 UNISOC

Codec2框架图

Unisoc Confidential For longcheer

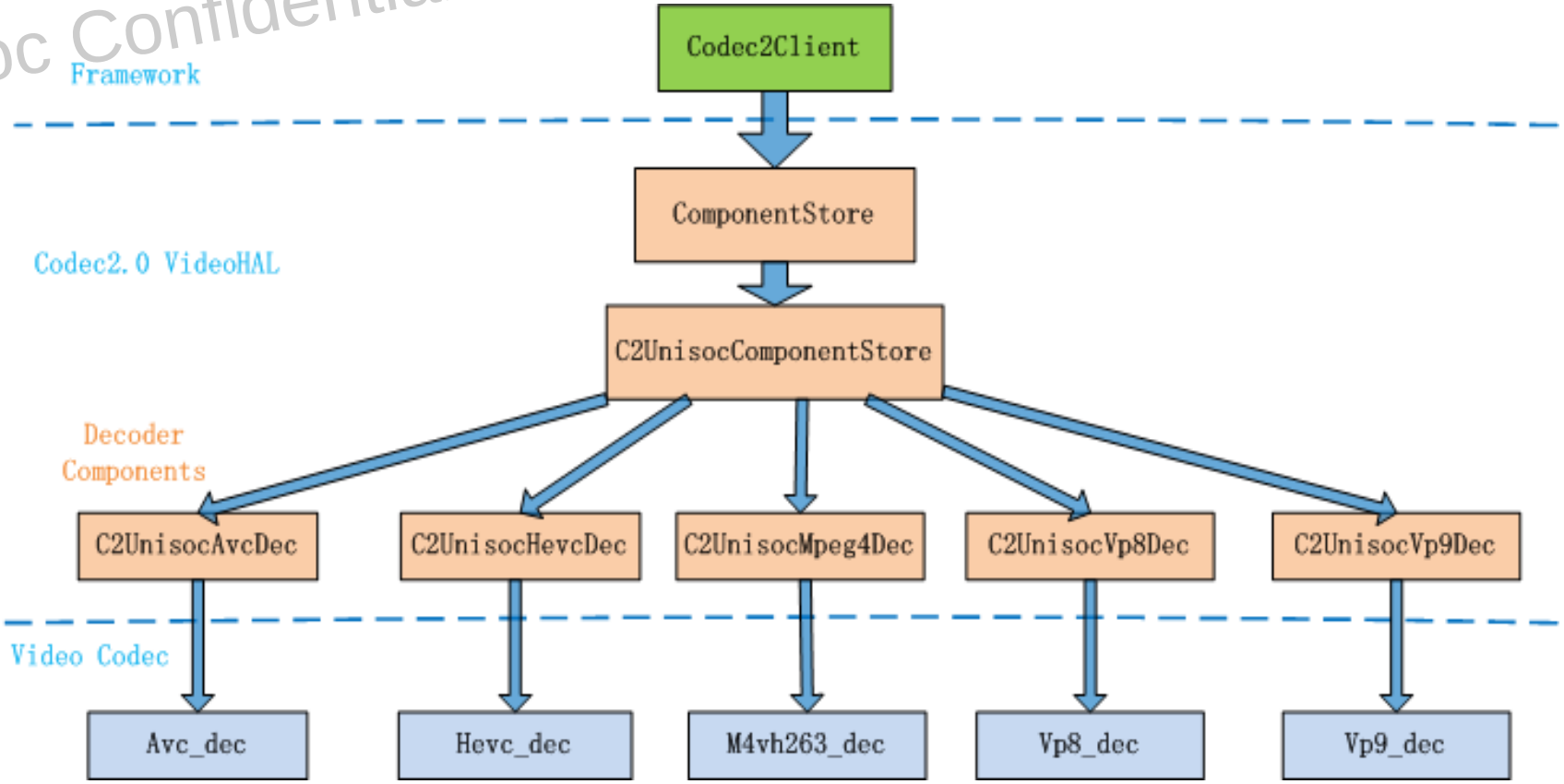


# Android平台音视频框架简单介绍

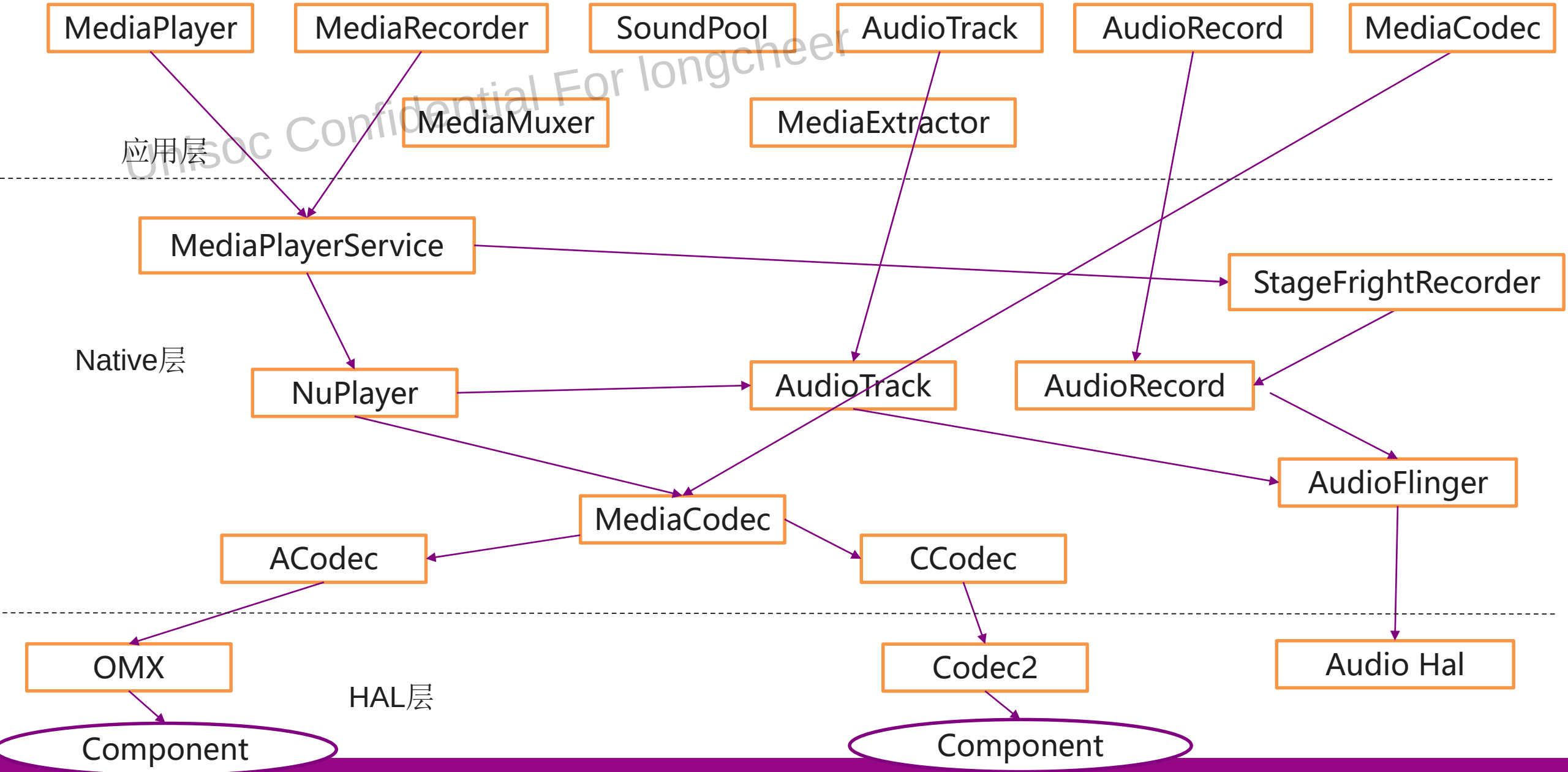
紫光展锐 UNISOC

平台Codec2组件框图

Unisoc Confidential For longcheer



# Android平台音视频框架简单介绍




Unisoc Confidential For longcheer

# Android平台音视频框架简单介绍

紫光展锐 UNISOC

平台代码位置简要说明：

Unisoc Confidential For longcheer

xref: /sprdroid12_trunk_22b/frameworks/base/media/java/android/media/

Home | Annotate Search current directory

| | | | | | |
|---|---|---|---|---|---|
| MediaMetadataRetriever.java | A D | 08-Jul-2022 | 55.9 KiB | 1,396 | 582 |
| MediaMetrics.java | A D | 08-Jul-2022 | 34.5 KiB | 834 | 457 |
| MediaMuxer.java | A D | 08-Jul-2022 | 28.8 KiB | 745 | 248 |
| MediaPlayer.java | A D | 08-Jul-2022 | 250.9 KiB | 6,253 | 3,070 |
| MediaRecorder.java | A D | 08-Jul-2022 | 83.4 KiB | 2,084 | 777 |

这里基本是提供给上层应用调用的Java接口

# Android平台音视频框架简单介绍

紫光展锐 UNISOC

平台代码位置简要说明：

Unisoc Confidential For longcheer

codec2 – C2框架
codecs – google的软解
extractors – 解析器
libaudioclient - audio client
libmediaplayerservice – 媒体服务
libstagefright - MediaCodec等相关框架

xref: /sprdroid12_trunk_22b/frameworks/av/media/

Home | Annotate Search current directory

| Name | Date | Size | #Lines | LOC |
|---|---|---|---|---|
| .. | 08-Jul-2022 | - | | |
| audioserver/ | 08-Jul-2022 | - | 293 | 232 |
| bufferpool/ | 08-Jul-2022 | - | 9,984 | 7,080 |
| codec2/ | 08-Jul-2022 | - | 112,510 | 84,402 |
| codecs/ | 08-Jul-2022 | - | 205,916 | 94,494 |
| common_time/ | 08-Jul-2022 | - | 2 | 1 |
| extractors/ | 08-Jul-2022 | - | 28,140 | 21,655 |
| img_utils/ | 08-Jul-2022 | - | 5,621 | 3,776 |
| janitors/ | 08-Jul-2022 | - | 11 | 9 |
| libaaudio/ | 08-Jul-2022 | - | 26,903 | 16,953 |
| libaudioclient/ | 08-Jul-2022 | - | 33,537 | 23,602 |
| libaudiofoundation/ | 08-Jul-2022 | - | 2,651 | 1,997 |
| libaudiohal/ | 08-Jul-2022 | - | 6,101 | 4,146 |
| libaudioprocessing/ | 08-Jul-2022 | - | 14,142 | 9,983 |
| libcpustats/ | 08-Jul-2022 | - | 609 | 384 |
| libdatasource/ | 08-Jul-2022 | - | 2,057 | 1,338 |
| libeffects/ | 08-Jul-2022 | - | 60,652 | 37,253 |
| libheif/ | 08-Jul-2022 | - | 1,026 | 646 |
| libmedia/ | 08-Jul-2022 | - | 32,929 | 25,999 |
| libmediaformatshaper/ | 08-Jul-2022 | - | 1,923 | 1,189 |
| libmediahelper/ | 08-Jul-2022 | - | 2,041 | 1,436 |
| libmediametrics/ | 08-Jul-2022 | - | 2,704 | 1,882 |
| libmediaplayerservice/ | 08-Jul-2022 | - | 28,286 | 21,324 |
| libmediatranscoding/ | 08-Jul-2022 | - | 17,318 | 11,327 |
| libnbaio/ | 08-Jul-2022 | - | 2,488 | 1,349 |
| libnblog/ | 08-Jul-2022 | - | 3,476 | 2,168 |
| libshmem/ | 08-Jul-2022 | - | 529 | 307 |
| libstagefright/ | 08-Jul-2022 | - | 169,371 | 122,958 |

# Android平台音视频框架简单介绍

紫光展锐 UNISOC

平台代码位置简要说明：

Unisoc Confidential For longcheer

xref: /sprdroid12_trunk_22b/frameworks/av/services/

Home | Annotate Search current directory

| Name | Date | Size | #Lines | LOC |
|---|---|---|---|---|
| .. | 08-Jul-2022 | - | | |
| audioflinger/ | 08-Sep-2022 | - | 34,224 | 25,211 |
| audiopolicy/ | 08-Jul-2022 | - | 60,799 | 48,978 |
| camera/ | 08-Jul-2022 | - | 67,243 | 48,746 |
| mediacodec/ | 08-Jul-2022 | - | 1,624 | 1,422 |
| mediaextractor/ | 08-Jul-2022 | - | 746 | 586 |

audioflinger - 上层音频与底层hal的一个桥梁
audiopolicy - 音频策略相关
mediaextractor - 解析器相关服务

# Android平台音视频框架简单介绍

紫光展锐 UNISOC

平台代码位置简要说明：

Unisoc Confidential For longcheer

xref: /sprdroid12_trunk_22b/vendor/sprd/modules/media/

Home | Annotate Search current directory

| Name | Date | Size | #Lines | LOC |
|---|---|---|---|---|
| .. | 08-Jul-2022 | - | | |
| codec2/ | 08-Jul-2022 | - | 35,619 | 27,683 |
| libstagefrighthw/ | 08-Jul-2022 | - | 10,200 | 5,274 |
| mediaAnalysis/ | 08-Jul-2022 | - | 74,392 | 67,666 |
| omx-components/ | 08-Jul-2022 | - | 31,847 | 24,224 |
| omx_core/ | 08-Jul-2022 | - | 410 | 295 |
| v2/ | 08-Jul-2022 | - | 42,707 | 29,307 |

音视频的编解码组件相关代码， 以前的 omx组件在v2目录中， 目前A12之后的版本视频编解码统一使用codec2框架，组件在codec2目录中

# Android平台音视频框架简单介绍

紫光展锐 UNISOC

平台代码位置简要说明：

Unisoc Confidential For longcheer

xref: /sprdroid12_trunk_22b/vendor/sprd/proprietories-source/sprd_codecs/

Home | Annotate Search current directory

| Name | Date | Size | #Lines | LOC |
|---|---|---|---|---|
| .. | 08-Jul-2022 | - | | |
| AudioDebug/ | 08-Jul-2022 | - | 9,870 | 8,206 |
| avc_h264_mix/ | 08-Jul-2022 | - | 105,894 | 81,881 |
| hevc_h265_mix/ | 08-Jul-2022 | - | 13,804 | 9,847 |
| jpeg/ | 08-Jul-2022 | - | 52,594 | 37,197 |
| m4v_h263_mix/ | 08-Jul-2022 | - | 44,808 | 32,147 |
| mp3dec/ | 08-Jul-2022 | - | 21,790 | 17,673 |
| mp3decg/ | 08-Jul-2022 | - | 462 | 213 |
| mp3deci/ | 08-Jul-2022 | - | 100,615 | 45,465 |
| mp3enc/ | 08-Jul-2022 | - | 19,045 | 15,393 |
| softav1/ | 08-Jul-2022 | - | 70,874 | 59,205 |
| softdec/ | 08-Jul-2022 | - | 31,784 | 11,189 |
| utest_vsp/ | 08-Jul-2022 | - | 20,321 | 15,240 |
| vp9_mix/dec/hw/ | 08-Jul-2022 | - | 14,566 | 10,491 |
| vpu_enc/ | 08-Jul-2022 | - | 18,253 | 13,096 |
| vpx_mix/ | 08-Jul-2022 | - | 14,444 | 9,870 |
| vsp/ | 08-Jul-2022 | - | 3,667 | 2,130 |

这部分代码是平台的编解码算法(闭源)。

紫光展锐 UNISOC

Unisoc Confidential For longcheer

# 06

# 常用音视频分析工具介绍

# 常用音视频分析工具介绍

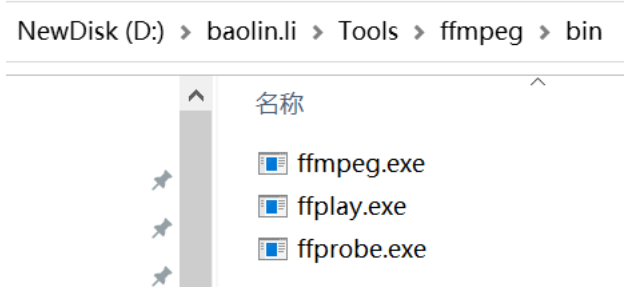
紫光展锐 UNISOC

**1.ffmpeg**

ffmpeg是一款开源的强大的音视频处理框架，市面上大部分的音视频相关应用都有他的身影，有些视频编解码不走平台的，可能就是封装了ffmpeg相关的编解码库。

Unisoc Confidential Longcheer

ffmpeg中带了几个工具，可以方便我们分析简单的音视频信息。




ffmpeg – 命令行工具， 可以做一些音视频处理， 比如说提取pcm、码流、YUV， 也可以做音视频合成，音视频分离，加水印等， 功能十分强大。

ffplay -- 播放器， 可以在命令行，敲入 ffplay + 文件路径， 直接播放视频

ffprobe – 音视频信息分析， 可以打印出音视频的编码类型，码率，分辨率等等信息。

# 常用音视频分析工具介绍

紫光展锐 UNISOC

Unisoc Confidential For longcheer

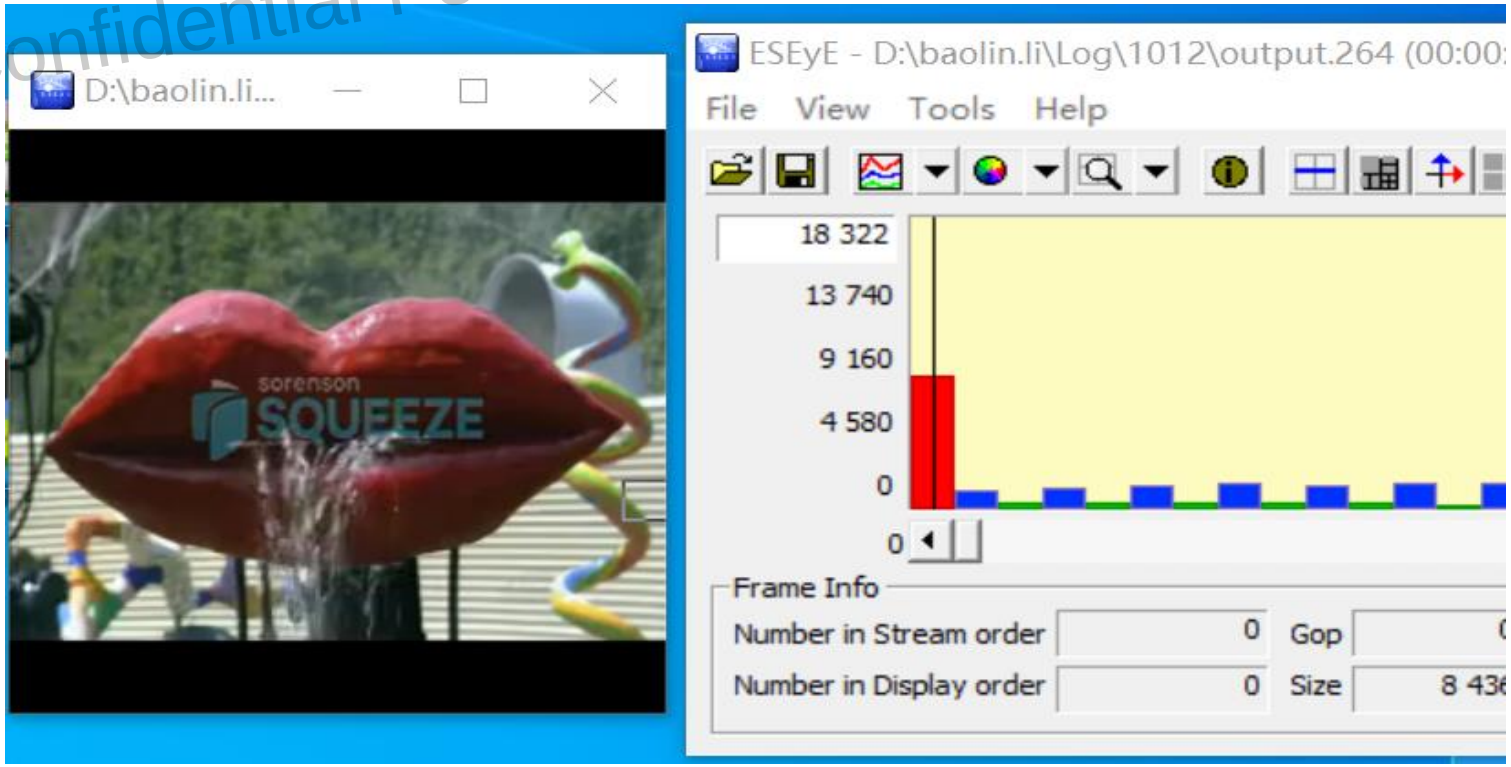
```
#提取yuv
ffmpeg -i input.mp4 -pix_fmt nv12  out.yuv

#提取h264码流
ffmpeg -i input.mp4 out.h264

#提取pcm, 采样率44100，通道数2， 格式:16位小端
ffmpeg -i input.mp4  -ar 44100 -ac 2 -f s16le  44100_2_s16le.pcm

#去除音频
ffmpeg -i 1.mp4 -c:v copy -an 2.mp4

#去除视频
ffmpeg -i 1.mp4 -c:v copy -vn 3.mp4

#裁剪视频， 从哪个时间点开始，持续多长时间
ffmpeg -ss 00:03:02 -t 00:01:00 -y -i 1.mp4 -vcodec copy -acodec copy 2.mp4
```

# 常用音视频分析工具介绍

Unisoc Confidential For longcheer

**2. streameye**

查看码流数据信息




红色的是关键帧 I 帧， 蓝色是 P 帧， 绿色是 B 帧

正常的视频，第一帧都是关键帧， 隔几帧会有一个关键帧。

有些视频关键帧间隔比较长， 就会出现seek操作的时候， 画面跳动很大， 因为必须找到一个关键帧，才能解码出一幅完整的图像。

# 常用音视频分析工具介绍

紫光展锐 UNISOC

**3. YUVPlayer**

查看YUV数据

Unisoc Confidential For longcheer

可以看YUV数据中是否存在花屏等问题， 其中 Size、Color 选项需要调整为当前YUV的参数， 如240x240大小， 颜色格式是 NV12

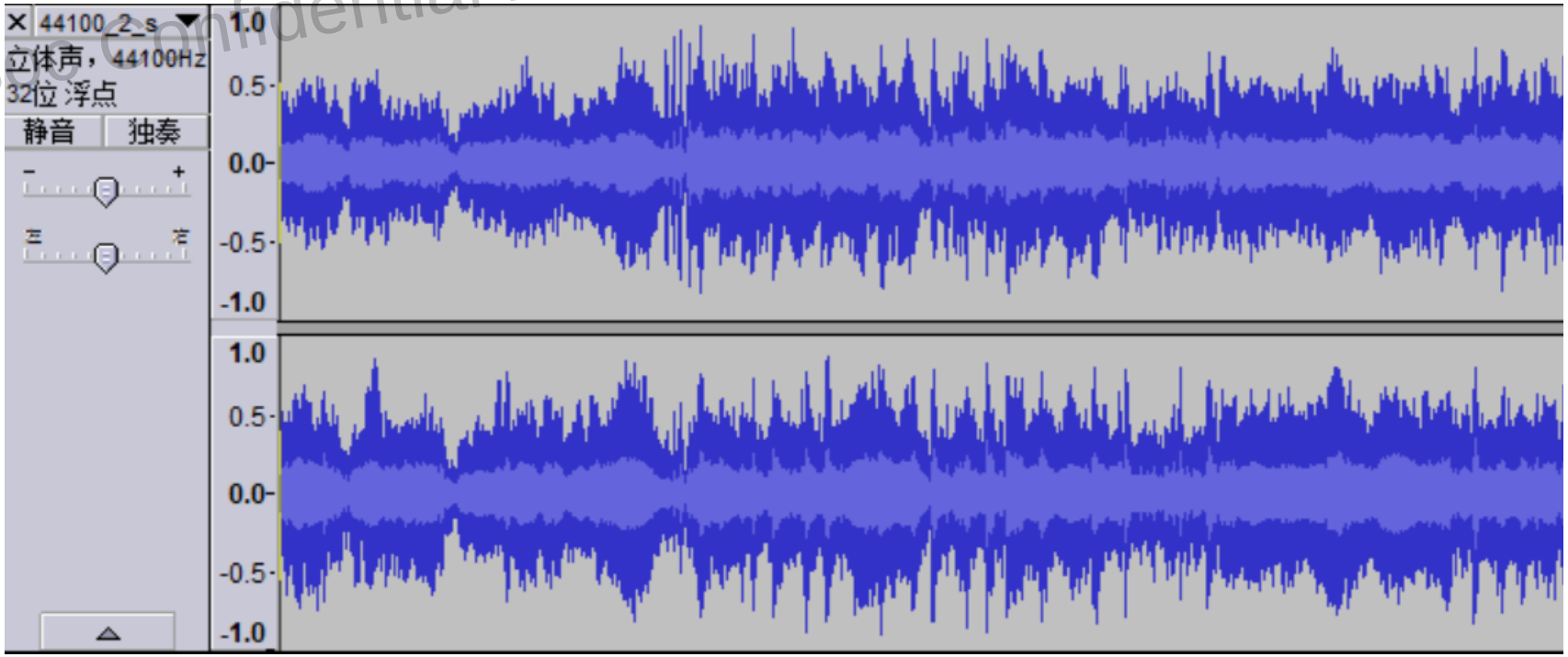
# 常用音视频分析工具介绍

紫光展锐 UNISOC

**4. audacity**

查看PCM数据

Unisoc Confidential For longcheer




用这个工具查看音频数据是否存在卡顿杂音问题， 并看是哪个阶段引入问题， 可能是应用下发的数据问题，也可能是hal算法处理后产生， 具体问题具体分析

# 谢 谢

本文件所含数据和信息都属于紫光展锐（上海）科技有限公司（以下简称紫光展锐）所有的机密信息，紫光展锐保留所有相关权利。本文件仅为信息参考之目的提供，不包含任何明示或默示的知识产权许可，也不表示有任何明示或默示的保证，包括但不限于满足任何特殊目的、不侵权或性能。当您接受这份文件时，即表示您同意本文件中内容和信息属于紫光展锐机密信息，且同意在未获得紫光展锐书面同意前，不使用或复制本文件的整体或部分，也不向任何其他方披露本文件内容。紫光展锐有权在未经事先通知的情况下，在任何时候对本文件做任何修改。紫光展锐对本文件所含数据和信息不做任何保证，在任何情况下，紫光展锐均不负责任何与本文件相关的直接或间接的、任何伤害或损失。
请参照交付物中说明文档对紫光展锐交付物进行使用，任何人对紫光展锐交付物的修改、定制化或违反说明文档的指引对紫光展锐交付物进行使用造成的任何损失由其自行承担。紫光展锐交付物中的性能指标、测试结果和参数等，均为在紫光展锐内部研发和测试系统中获得的，仅供参考，若任何人需要对交付物进行商用或量产，需要结合自身的软硬件测试环境进行全面的测试和调试。

紫光展锐、UNISOC、、展讯、Spreadtrum、SPRD、锐迪科、RDA及其他紫光展锐的商标均为紫光展锐（上海）科技有限公司及/或其子公司、关联公司所有。
**Bluetooth**®文字商标和徽标为蓝牙技术联盟的注册商标，紫光展锐（上海）科技有限公司对此类商标的任何使用均已获得许可。
本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

本文档可能包含第三方内容，包括但不限于第三方信息、软件、组件、数据等。紫光展锐不控制且不对第三方内容承担任何责任，包括但不限于准确性、兼容性、可靠性、可用性、合法性、适当性、性能、不侵权、更新状态等，除非本文档另有明确说明。在本文档中提及或引用任何第三方内容不代表紫光展锐对第三方内容的认可、承诺或保证。
用户有义务结合自身情况，检查上述第三方内容的可用性。若需要第三方许可，应通过合法途径获取第三方许可，除非本文档另有明确说明。